Assignment 1 CS170: Introduction to Artificial Intelligence, Dr. Eamonn Keogh

Yiyang Chen
SID X746706
Email ychen1420@ucr.edu
Date Feb-13-2026

In completing this assignment I consulted:

- The Blind Search and Heuristic Search lecture slides and notes annotated from lecture.
- Python 3.6 Documentation. This is the URL to the Table of Contents of 3.6.15: https://docs.python.org/3.6/
- For the randomly generated puzzles: https://sliding.toys/mystic-square/8-puzzle/

All important code is original. Unimportant subroutines that are not completely original are…

- Instead of using heapq, I manually implemented a search loop to find minimum f(n) in the frontier list.
- I imported the time library, used the time.time( ) function to measure the search time.
- I used my custom copy method in Puzzle class using row.copy( ), to ensure the state change do not affect any parent nodes.
- I use a python lambda function to define the heuristic function h(n)=0 for uniform cost search, using lambda state:0. URL: https://www.w3schools.com/python/python_lambda.asp
- All data visualizations in this report were plotted using Origin 2025 (Student Version)

Outline of this report:

- Cover page: (this page)
- My report: Pages 2 to 13.
- Sample trace on an easy problem, page 14.
- Sample trace on a hard problem, page 15.
- My code pages 16 to 21. Note that in case you want to run my code, here is a URL that points to it online: https://github.com/cyy-07/UCR-CS170-Project1_8puzzle

CS170: Assignment 1: The eight-puzzle

Yiyang Chen, SID X746706 Feb-13-2026

Introduction

The 8-puzzle problem, also known as a special form of "sliding block puzzle[1]", often appears when learning search methods in artificial intelligence class. Since I was a young kid, my grandmother taught me how to play Klotski[2] , which is also a sliding block puzzle, shown in Figure 2. Both the Eight Puzzle and Klotski have a same core feature: tiles must be moved through an empty space. When I played it as a child, I always wondered: "Is there any way to solve it perfectly, without repeated moves?" Several years later, I realized this curiosity was actually the classic problem of state space search. The 8-puzzle is 3×3 in grid size with uniform 1×1 tiles from 1 to 8 and a blank. The rule is also quite simple: We start from a disordered initial state, and try to reach an orderly goal state. We can slide any tile next to the blank grid, which creates a new blank in the original position. (I regard it as a swop between the specific tile and the blank in my project). Mathematically, the essence of the search problem is to find a shortest path from the initial state to the target among millions of possibilities(among the searching space).

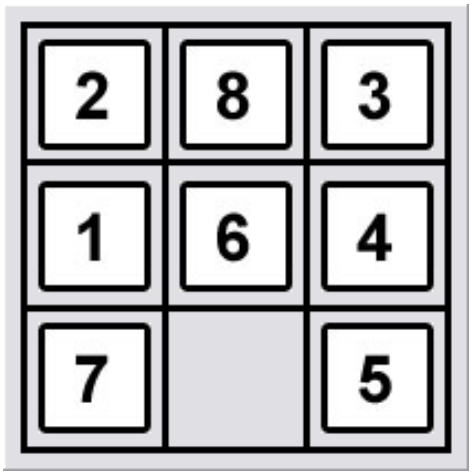


Figure1: Initial state of 8-puzzle problem[3]

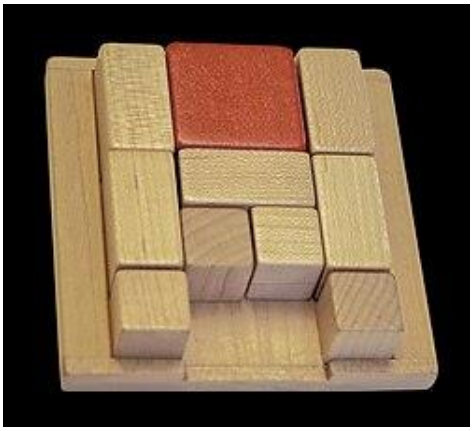


Figure2: The layout of Klotski puzzle

"In a Klotski puzzle, the largest block must be moved to the bottom middle location so that it can be slid over the

[1] The explanation of sliding block problem in Wikipedia: https://en.wikipedia.org/wiki/Sliding_puzzle
[2] The picture and explanation of Klotski problem in Wikipedia: https://en.wikipedia.org/wiki/Klotski
[3] The picture is from: https://www.8puzzle.com/8_puzzle_problem.html

	border, without any of the other blocks being removed in this way." [4]

This project implements three search algorithms to solve the 8-puzzle problem. The three ways are : Uniform Cost Search, A* with Misplaced Tile heuristic, and A* with Manhattan Distance heuristic. To explore these search strategies, I completed this project, as part of Dr. Eamonn Keogh's Introduction to AI(CS170) course at UC Riverside (Winter 2026). I chose Python3 as my programming language mainly because of its readability and simplicity, making it easier for me and graders to read, even though it is not so fast.

Comparison of Algorithms

In this project, I implemented and compared three different search strategies. The three algorithms are: Uniform Cost Search, Misplaced Tile Heuristic with A* search, and the Manhattan Distance Heuristic with A* search. Each algorithm was implemented on same foundation, and differs in the heuristic function, which is related to the node expansion. The goal of the comparison is to check how different heuristic function affects the efficiency in order to find the optimal solution.

Uniform Cost Search

As we know in the class and according to Dr. Eamonn's slides, "*Uniform Cost Search expands nodes in order of cumulative path cost g(n), and is complete and optimal under nondecreasing path costs* [5]." It does not use any heuristic information, so we can view it as a special case of A* search with the heuristic function h(n)= 0. As a result, node expansion is determined simply by the path cost g(n), making the evaluation function f(n)=g(n). Therefore, in my project (search.py file, uniform_cost_search function), I use lambda[6] to create an anonymous function that for any state always hardcodes to return 0. In my project, g(n) path cost stands for the required number of moves to reach the current state from the starting state (each move has the same cost 1). The main logic for me is exploring the state space in a similar way like breadth first search order, iterating all the nodes and finally achieving the final state.

The Misplaced Tile Heuristic

The misplaced tile heuristic is a way to calculate the h(n) value by counting how many tiles are in the "wrong" position. If the current state position is different from the goal state, we simply count the misplaced tiles and return the number, passing

[4] The picture and explanation of Klotski problem in Wikipedia: https://en.wikipedia.org/wiki/Klotski
[5] From Dr. Eamonn's slide:
[6] Lambda function: https://www.w3schools.com/python/python_lambda.asp

the h(n) into the function in our search. As shown in Figure 3, if we consider the initial state, we are able to identify the tile 4, 5 and 6 are misplaced. Since the blank tile (0) is not counted, the total heuristic value is h(n)= 3. As outlined in the course slides, the blank tile (we implement as 0 or * in the python file) is excluded from this count to ensure the heuristic remains admissible[7]. It also guarantees that h(n) would never be larger than the real number of moves needed to win.

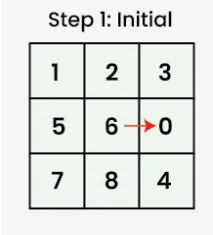


Figure 3: An example of misplaced count = 3[8]

In my project (heuristics.py file), I directly use for loops to iterate through the whole 3*3 grid, with the variable 'count' updating in the count_misplaced_tile function. After combining h(n) and g(n), the state with the smallest amount of f(n) value is expanded first. The A* search will always pick the state with the smallest f(n), which allow a more efficient path to the goal state, compare to the blind search.

The Manhattan Distance Heuristic

The Manhattan distance is a more advanced heuristic, compared to the two search strategies we just introduced. It introduces a different way for us to calculate the cost of each state. The distance for a tile is the number of horizontal and vertical moves that we need to reach its goal state. For example, as shown in Figure 3, we can see how it works: For the number (4), it is at (2, 2), but its goal is (1, 0). This tile is far away! It needs 1 move up and 2 moves left, which is 3 moves in total. The tile 5 and 6 both need 1 move. Therefore, by summing these up, the total h(n) becomes 5. In the misplaced tile version, this same state only had an h(n) of 3. We cannot simply say this one is better than the other, because calculating the Manhattan Distance takes more computation time for the computer in each step, which we will analyze in next part. Finally, A* would use the f(n) value we calculated and search for the final result.

In my project, I implemented the Manhattan Distance heuristic in the manhattan_distance( ) function in heuristics.py. I create a dictionary(goal_pos) to store the goal position, so that every time when calculating distance, it only takes

[7] According to Dr. Eamonn's slides, "admissible" means that it never overestimates the cost to reach the goal.https://www.dropbox.com/scl/fo/blbkjaf1eyl94lij5wl2b/AAvNKn0YrnaX0oGPQn7ueFo?rlkey=alq2gb2ftsw73hcar4lk897r0&e=3&dl=0 03-Heuristic Search, page 38

[8] Figure 3 is from: https://www.geeksforgeeks.org/dsa/8-puzzle-problem-using-branch-and-bound/

O(1) time to look up. Then I iterate each tile in the current state and calculate both horizontally and vertically. Finally abs( ) function is used to make sure all distances are positive. With all three algorithms implemented, I will next compare their performance on puzzles of varying difficulty.

Comparison of Algorithms on Sample Puzzles and Results

As shown in the Figure 4, some tests examples of different depth ranging from depth 0 to depth 24 are used as my way to test the performance of all three algorithms. To better evaluate each algorithm, I tried to compare them in three dimensions: (1) the number of nodes expanded, (2) the maximum queue size, and (3) the running time.



Figure 4: Test example of different depth[9]

The number of expanded nodes can reflect how many states an algorithm has to explore before getting to goal state. Table 1 presents the results on Dr. Eamonn's test example on different algorithm. Firstly, we compare expanded nodes, which indicate the efficiency of each heuristic function.

Depth	UCS	Misplaced tiles	Manhattan Distance
0	0	0	0
2	3	2	2
4	27	4	4
8	217	18	12
12	1769	121	36
16	11399	696	96
20	54373	3622	531
24	234994	22775	1869

Table 1: Comparison of nodes expanded across different search algorithms at increasing depths

[9] From Dr. Eamonn's test example.

Then, in order to visualize the results effectively, I used Origin 2025 for plotting bar chart.

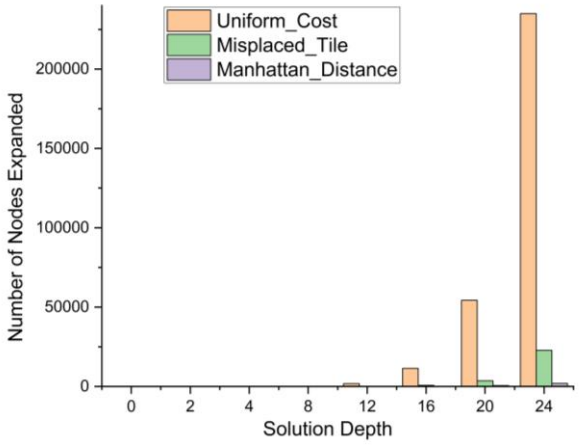


Figure 5: The comparison of node expansion in bar chart

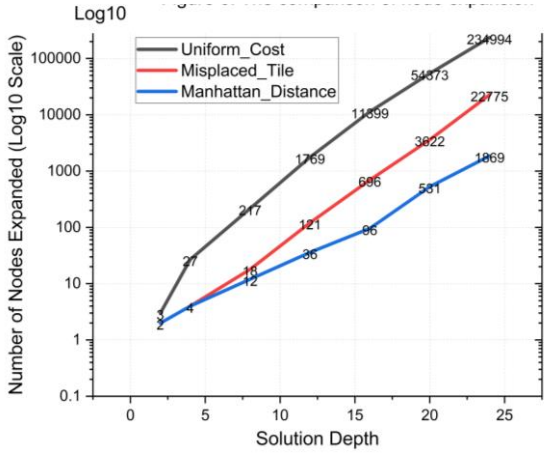


Figure 6: The comparison of node expansion in line graph in semilog scale

Figure 5 shows the comparison of node expansion with the solution depth increasing. we can easily observe significant differences in algorithm performance. However, if the depth is 0 to 16 (easier puzzles), we can hardly tell the difference between different algorithm. Due to the exhaustive exploration by the uniform cost search in the state space, vertical axis must accommodate nearly 240,000 nodes at depth 24. Therefore, I replaced the bar chart with a line graph. Line graphs are better for showing the rate of change and growing speed as problem complexity increases. But due to the explosion of nodes in the uniform cost search, the other two lines are almost flat near the x-axis, making it hard to compare. Furthermore, to solve the scaling issue, I applied a logarithmic scale(log10) to the y-axis, called *semilog plot*[10], shown in Figure 6.

From the table and figures above, we can conclude some insights first: (1) For all algorithms, they all demonstrate positive correlations between solution depth and node expansion. (2) Even when on a semilog plot, the Manhattan distance heuristic consistently expands the fewest nodes. (3) The slope of the lines in Figure 6 reveals the growth rate of each algorithm. Manhattan distance maintains the flattest slope, showing that it can provide us with much more efficiency in heuristic function h(n) in estimating the remaining cost. A more detailed summary of these conclusions will be shown in the conclusion section.

[10] This approach is widely used in my major, chemical engineering, especially when plotting the relationship between the Reynolds Number and the friction factor for fluids (e.g., the Moody chart). https://en.wikipedia.org/wiki/Moody_chart

Secondly, it is vital for us to compare the maximum frontier queue size. In searching algorithms in CS field, the frontier stands for all nodes that have been generated but not yet expanded. Therefore, the maximum frontier size directly reflects the memory consumption of an algorithm. In my implementation within search.py file, at each iteration of the search loop, the algorithm checks the current length of the frontier and updates the max_queue_size variable. As shown in table 2, the results indicate the space efficiency among the three algorithms.

Depth	UCS	Misplaced tiles	Manhattan Distance
0	1	1	1
2	4	3	3
4	22	6	6
8	150	16	12
12	1157	90	28
16	7345	451	64
20	29404	2219	346
24	71269	11494	1098

Table2: Comparison of maximum frontier queue size (maximum memory consumption) across increasing solution depths

As shown in Figures 7 and 8, we can observe similar trends when comparing the maximum frontier queue size. Similarly, as the solution depth increases, all three algorithms require more memory to store generated nodes.

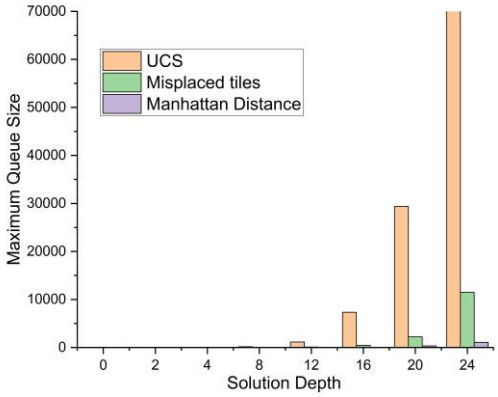


Figure 7: Comparison of maximum frontier queue size across different search algorithms

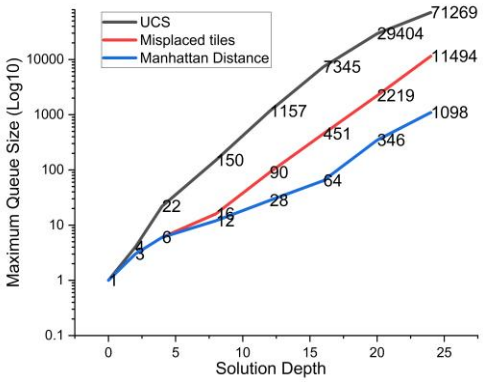


Figure 8: Comparison of maximum frontier queue size across different search algorithms in semilog scale

For the uniform cost search, it must explore a large number of intermediate states due to heuristic function h(n)=0. So before reaching final goal state, it is necessary for the algorithm to store them in the frontier, resulting in a huge queue size. In contrast, A* search with heuristic functions demonstrates better performance. Meanwhile, A* search also depends on how informative the heuristic function can be. In our example, we can conclude that in this special 8-puzzle problem, using Manhattan distance as h(n) is more informative than misplaced tile. In summary, uniform cost search still shows the fastest growing trend, while Manhattan distance maintains the slowest increase, showing the best performance.

Finally, the third part of this performance evaluation is total execution time. Execution time is an important metric in CS because it directly reflects the computational time cost, which directly determines whether an algorithm is practical or not. As shown in table 3, I imported the *time module*[11] to record the time of all three algorithms. During the process of measuring the time, I noticed that when the solution depth was small (depth = 0 to 8), recording the running time with only one decimal place, would result in time displaying as 0.0 seconds. Therefore, for solution depths ranging from 0 to 8, I intentionally recorded with two decimal places in the result, in order to provide better observation of slightly time differences. However, it should also be noted that even when two decimal places are presented, the last digit still remains uncertainty, because it is obtained through rounding by the computer[12].

Depth	UCS	Misplaced tiles	Manhattan Distance
0	0.00	0.00	0.00
2	0.01	0.01	0.01
4	0.03	0.01	0.01
8	0.30	0.04	0.02
12	2.7	0.2	0.1
16	25.8	1.0	0.1
20	328.1	6.5	0.7
24	4182.0	93.8	2.6

Table 3: Comparison of total execution time in seconds across increasing solution depths

With the increase in solution depth, the running time of all three algorithms also rises noticeably. Although the small solution depths do not show much of a difference, the performance differs drastically as the depth grows, especially in depth 20 and 24. Uniform Cost Search always took the longest execution time, and also demonstrates the most dramatic increase in execution time as the solution

[11] Python time library: https://docs.python.org/3/library/time.html
[12] Precision limitations in computer: https://floating-point-gui.de/

depth grows. In contrast, both A* search perform quite better. A* with Manhattan distance still performs best because it provides more accurate estimation of the remaining cost. This suggests that in the case of the 8-puzzle, the Manhattan distance heuristic is the most effective one compared to another two algorithms, because it reduces both the number of expanded nodes and the memory and is both effective in space and time complexity.

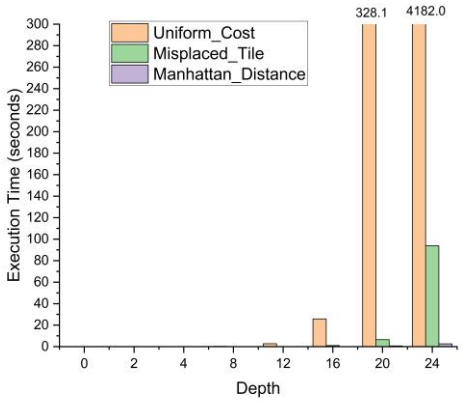


Figure 9: Comparison of execution time across different search algorithms in bar chart

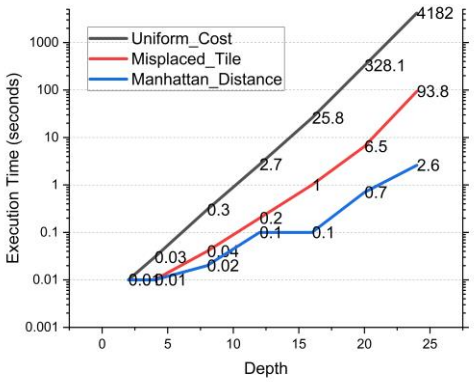


Figure 10: Comparison of execution time across different search algorithms in semilog scale

Additionally, I think it is worth to mention that overall execution time of the algorithms seems relatively long. Besides the natural growth of the search space with increasing depth, I inferred two other factors that would affect the total time. First, I maintained the frontier using a standard list, instead of a priority queue, which would result in O(n) time complexity. When the frontier becomes large, total time would differ compared to O(logn) in *heapq method*[13]. Another reason might be hardware performance limitations. The tests were conducted on my windows laptop that has been used for four years. I am sure that different laptop and computing environments would have different time results, but the overall increasing trends should be identical.

In summary, the results from my tests across node expansion, memory consumption, and execution time all consistently demonstrate the effectiveness level of heuristic.

[13] The heapq method: https://docs.python.org/3/library/heapq.html

Additional Examples

I also created some different custom puzzle that I obtained resources from *the Internet*[14]. These puzzles have different initial states and solution depths. The purpose of including these additional examples is to verify whether the trends remain the same under different test conditions. Figure 11 shows all the additional examples.

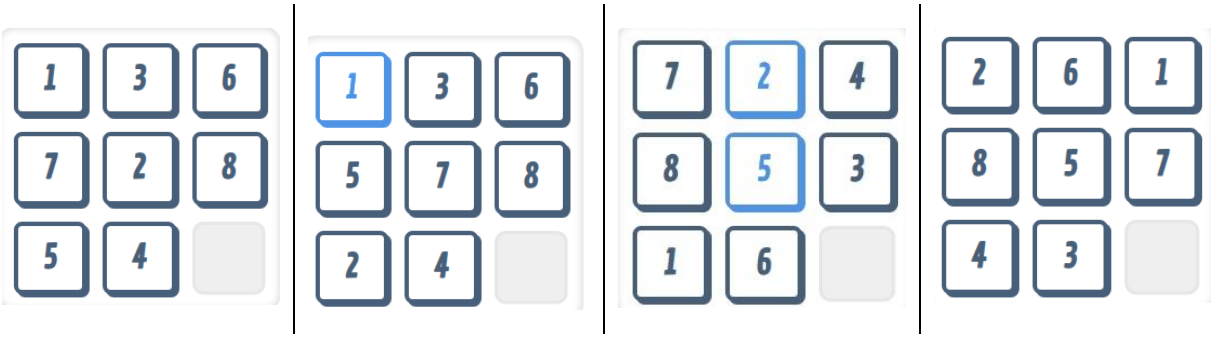


Figure 11: Additional examples of different solution depths *from online resources* [15](with solution depth in increasing order)

Depth	Nodes Expanded	Max Queue Size	Running time
10	11	11	0.02
16	161	110	0.23
18	133	92	0.15
24	2673	1570	5.27

Table 4: Performance of A* with Manhattan distance on additional online examples

For simplicity and fast validation, only A* search with the Manhattan distance heuristic is reported here. Follow the same procedure before, I drew a line graph to demonstrate. As shown in Figure 12, it still remains increasing trend as the solution depth grows, but the puzzle with depth 18 expands fewer nodes than depth 16. After carefully analyzing the reason, I concluded that: Node expansion is not only solely determined by solution depth, but is also greatly influenced by the specific puzzle. If there were some puzzle states, the search might use a more direct search path, which resulted in fewer expanded nodes.

[14] The website provides various puzzle example: https://sliding.toys/mystic-square/8-puzzle/
[15] Figures from the website: https://sliding.toys/mystic-square/8-puzzle/

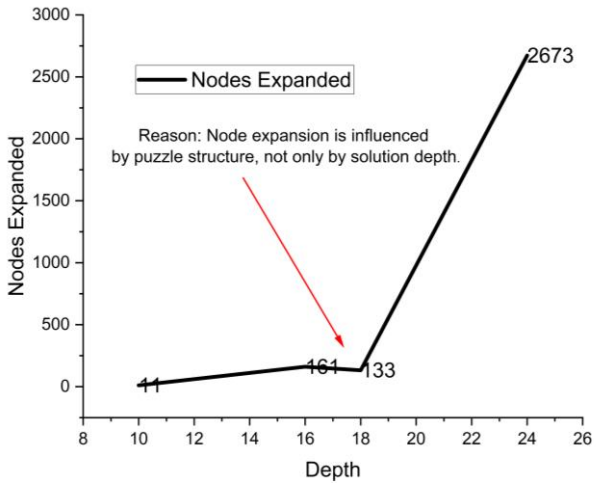


Figure 12: Node expansion of A* with Manhattan distance on additional puzzles

All in all, all these additional examples demonstrate that the performance trends remain consistent with my former examples. Also, they further support my conclusions drawn from the last session and show the robustness of my project.

Supporting Information

To better demonstrate my code structure, I created an additional slide summarizing the code structure. The slide summarizes the design of the project, including search logic, state representation, and heuristic functions. The link is provided here:

https://docs.google.com/presentation/d/1BefclDz4Z9awCzmYxW8Rv6ogEtsMADve/edit?usp=sharing&ouid=102171895618004307158&rtpof=true&sd=true

A short demonstration video is also provided. The link is provided here:

https://drive.google.com/drive/folders/1eLA719-lLIpwbELnpFQywWS1DBdg2Rch?usp=sharing

The Origin 2025 file for plotting is available:
https://drive.google.com/file/d/1Q0LEcNCh8AvGJjdvSfiZYQfPugCmfz6W/view?usp=sharing
The structure was inspired by Dr. Eamonn's pseudocode, and a website: https://www.educative.io/answers/how-to-solve-the-8-puzzle-problem-using-the-a-star-algorithm. Other online resources are provided:
For keep track of repeated state:
https://ai.stackexchange.com/questions/7555/how-do-i-keep-track-of-already-visited-states-in-breadth-first-search
For understanding of A* algorithm (from UCB, implemented in java):

https://www.redblobgames.com/pathfinding/a-star/introduction.html
For coming up with general idea:
https://blog.goodaudience.com/solving-8-puzzle-using-a-algorithm-7b509c331288

Reflection and Future Improvement

Although basic invalid input checking was implemented, such as the incorrect number when choosing menus, the custom puzzle input section still has lots of flaws. For example, as shown in Figure 13 and 14, when I enter my own puzzle, if I input numbers outside the range 0–8, or if I enter only one or four numbers in a row instead of three, or if I accidentally repeat a number, the code will completely crash.

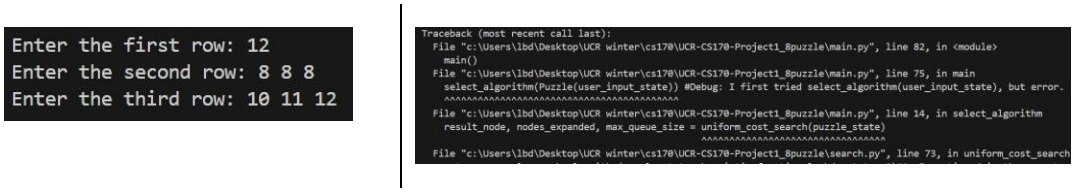


Figure 13&14: Example of a program crash caused by invalid input

Also, as I mentioned in the "Comparison of Algorithms on Sample Puzzles and Results" session, the current implementation uses a list to maintain the frontier. It has O(n) running time, which is quite inefficient comparing to use a priority queue (like heapq method).

Therefore, I came up with several extensions that could further make this project for the better: Firstly, instead of using numeric tiles, the puzzle could be visualized by *splitting an image into 9 equal parts*[16]. Secondly, at the very beginning of the report, I mentioned the Klotski puzzle, which includes blocks of different sizes, making the state representation even harder. Finally, a beautifully front end is needed for a good project. Although I am a non-CS major, lack of training and know nothing about the front-end developing, I am confident that *generative AI tools*[17] could assist front-end development in the future improvement.

Conclusion

Considering the list of the three algorithms and the comparisons between them: Uniform Cost Search, Misplaced Tiles, and Manhattan Distance, it can be concluded that:

[16] Inspired by the website here: https://www.slidingpuzzle.online/. From this website users are able to upload a specific picture and it can automatically splitting an image into 9 equal parts
[17] https://en.wikipedia.org/wiki/Generative_artificial_intelligence

- From data across all metrics, the Manhattan Distance heuristic performs as the superior algorithm and the best among the three tested algorithms for the 8-puzzle, followed by the Misplaced Tiles heuristic, and finally Uniform Cost Search, which is exhaustive, very slow and computationally expensive due to its lack of heuristic.
- The Uniform cost search effectively became a Breadth-First Search because UCS treats all paths equally (h(n)=0). It is actually limited by exponential time and space complexities O(b^d), and this exponential requirement becomes the primary obstacle especially when depth increases. This is why we can see such staggering figures in the UCS data when solution depth is large.
- While both heuristic functions improve the performance of A*, their effectiveness still depends on how "informative" the heuristic is. The Manhattan distance heuristic provides a better and more accurate estimation of the remaining cost, comparing to the Misplaced Tiles heuristic.
- It should also be noted that in some special cases, we observed a puzzle with a larger solution depth required fewer node expansions and shorter running time compared to another puzzle with a smaller depth. This phenomenon shows that the node expansion and execution time are not solely determined by solution depth. Instead, they are also influenced by the specific special structure of the puzzle. However, despite these small fluctuations, the overall performance trend remains the same, shown in the first point of the conclusion

The following is a traceback of an **easy** puzzle:

```
Welcome to Yiyang's 8-puzzle solver!
It is your choice now! If you choose "1", it will be a default test puzzle, if
 you choose "2", you can create your own puzzle!
Please enter 1 or 2: 2

It is your choice now! Please enter your own puzzle state, using 0 to represen
t the blank tile.
Please only enter valid 8-puzzles!
Type RETURN only when finished.

Enter the first row: 1 2 3
Enter the second row: 4 0 6
Enter the third row: 7 5 8

 Now, please select the algorithm you want to use to solve the puzzle.
choose '1' -> Uniform Cost Search
choose '2' -> A* with Misplaced Tile Heuristic
choose '3' -> A* with Manhattan Distance Heuristic
choose '4' -> Return to main menu(change puzzle)
Please enter a number from 1 to 4: 3
The best state to expand with g(n) = 0 and h(n) = 2 is...
1 2 3
4 * 6
7 5 8

The best state to expand with g(n) = 1 and h(n) = 1 is...
1 2 3
4 5 6
7 * 8

The best state to expand with g(n) = 2 and h(n) = 0 is...
1 2 3
4 5 6
7 8 *

Goal!!!
Solution depth was 2
Number of nodes expanded: 2
Max queue size: 5
Running time: 0.00 seconds
```

The following is a traceback of a **hard** (depth 16) puzzle.

```
Welcome to Yiyang's 8-puzzle solver!
It is your choice now! If you choose "1", it will be a default test puzzle, if
 you choose "2", you can create your own puzzle!
Please enter 1 or 2: 2

It is your choice now! Please enter your own puzzle state, using 0 to represen
t the blank tile.
Please only enter valid 8-puzzles!
Type RETURN only when finished.

Enter the first row: 1 6 7
Enter the second row: 5 0 3
Enter the third row: 4 8 2

 Now, please select the algorithm you want to use to solve the puzzle.
choose '1' -> Uniform Cost Search
choose '2' -> A* with Misplaced Tile Heuristic
choose '3' -> A* with Manhattan Distance Heuristic
choose '4' -> Return to main menu(change puzzle)
Please enter a number from 1 to 4: 3
The best state to expand with g(n) = 0 and h(n) = 12 is...
1 6 7
5 * 3
4 8 2

The best state to expand with g(n) = 1 and h(n) = 11 is...
1 6 7
* 5 3
4 8 2

......
......              \\ Here I delete about 10 pages of the trace to save space

The best state to expand with g(n) = 15 and h(n) = 1 is...
1 2 3
4 5 6
7 * 8

The best state to expand with g(n) = 16 and h(n) = 0 is...
1 2 3
4 5 6
7 8 *

Goal!!!
Solution depth was 16
Number of nodes expanded: 96
Max queue size: 64
Running time: 0.12 seconds
```

URL to my code is https://github.com/cyy-07/UCR-CS170-Project1_8puzzle, with 10 days evolving code available.

main.py

```
from puzzle import Puzzle
from search import uniform_cost_search, a_star_with_misplaced_tile, a_star_with_manhattan
from heuristics import uniform_cost, count_misplaced_tile, manhattan_distance
def select_algorithm(puzzle_state):
        while True:
            print("\n Now, please select the algorithm you want to use to solve the puzzle.")
            print("choose '1' -> Uniform Cost Search")
            print("choose '2' -> A* with Misplaced Tile Heuristic")
            print("choose '3' -> A* with Manhattan Distance Heuristic")
            print("choose '4' -> Return to main menu(change puzzle)")

            algorithm_choice = input("Please enter a number from 1 to 4: ")
            if algorithm_choice == "1":
                result_node, nodes_expanded, max_queue_size = uniform_cost_search(puzzle_state)
                break
            elif algorithm_choice == "2":
                result_node, nodes_expanded, max_queue_size = a_star_with_misplaced_tile(puzzle_state)
                break
            elif algorithm_choice == "3":
                result_node, nodes_expanded, max_queue_size = a_star_with_manhattan(puzzle_state)
                break
            elif algorithm_choice == "4":
                break
            else:
                print("Invalid choice! Please enter a number from 1 to 4.")
                continue
def main():
    while True:
        print("Welcome to Yiyang's 8-puzzle solver!")
        print("It is your choice now! If you choose \"1\", it will be a default test puzzle, if you choose \"2\",
 you can create your own puzzle!")
        user_choice = input("Please enter 1 or 2: ")
        if user_choice == "1":
            print("\n Default puzzles:(1-8, with increasing difficulty, which means the depth increasing)")
            print("choose '1' -> depth=0")
            print("choose '2' -> depth=2")
            print("choose '3' -> depth=4")
            print("choose '4' -> depth=8")
            print("choose '5' -> depth=12")
            print("choose '6' -> depth=16")
            print("choose '7' -> depth=20")
            print("choose '8' -> depth=24")
            difficulty_choice = input("Please enter a number from 1 to 8: ")
            # Dr. Eamonn mentioned in the report that we can use his test cases.
            if difficulty_choice == "1":
                test_state = [  [1,2,3], [4,5,6], [7,8,0]]#depth=0
            elif difficulty_choice == "2":
                test_state = [  [1,2,3], [4,5,6], [0,7,8]]#depth=2
            elif difficulty_choice == "3":
                test_state = [  [1,2,3], [5,0,6], [4,7,8]]#depth=4
            elif difficulty_choice == "4":
                test_state = [  [1,3,6], [5,0,2], [4,7,8]]#depth=8
            elif difficulty_choice == "5":
                test_state = [  [1,3,6], [5,0,7], [4,8,2]]#depth=12
            elif difficulty_choice == "6":
                test_state = [  [1,6,7], [5,0,3], [4,8,2]]#depth=16
            elif difficulty_choice == "7":
                test_state = [  [7,1,2], [4,8,5], [6,3,0]]#depth=20
            elif difficulty_choice == "8":
                test_state = [  [0,7,2], [4,6,1], [3,5,8]]#depth=24
            else:
                print("Invalid choice! Please enter a number from 1 to 8.")
                return
            select_algorithm(Puzzle(test_state))#Debug: I first tried select_algorithm(test_state), but error.
        elif user_choice == "2":
            print("\nIt is your choice now! Please enter your own puzzle state, using 0 to represent the blank t
ile.")
            print("Please only enter valid 8-puzzles!")
            print("Type RETURN only when finished.\n")
            own_puzzle_row_one = input("Enter the first row: ")
            own_puzzle_row_two = input("Enter the second row: ")
```

```
            own_puzzle_row_three = input("Enter the third row: ")
            own_puzzle_row_one =[int(x) for x in own_puzzle_row_one.split()]
            own_puzzle_row_two =[int(x) for x in own_puzzle_row_two.split()]
            own_puzzle_row_three =[int(x) for x in own_puzzle_row_three.split()]
            user_input_state = [own_puzzle_row_one, own_puzzle_row_two, own_puzzle_row_three]
            select_algorithm(Puzzle(user_input_state)) #Debug: I first tried select_algorithm(user_input_state),
 but error.
            # Fixed by wrapping the list into a Puzzle object so the search algorithm so that we can use .displa
y()
        else:
            print("Invalid choice! Please enter 1 or 2.")
            return

if __name__ == "__main__":
        main()
```

heuristic.py

```
#In this heuristics.py file, we define different heuristic functions for the 8-puzzle problem.
#Let's start with the uniform cost search. It is quite simple because h_n is always 0.
#Therefore, f_n=g_n so we only care about how far we need to reach the node.

from puzzle import Puzzle
from node import Node
def uniform_cost(state):
    return 0

#Now we define the misplaced tile heuristic search. It is also simple because we only count the number of mispla
ces tiles.
def count_misplaced_tile(state):
    goal = [[1,2,3],
            [4,5,6],
            [7,8,0]]
    count = 0
    for i in range(3):
        for j in range(3):    #count how many tiles are wrong placed.
            if state[i][j] != 0 and state[i][j] != goal[i][j]:
                count += 1
    return count

#Finally, we define the Manhattan distance search.
#Manhattan distance is the sum of the vertical and horizontal distances(absolute value) from current position to
 the goal for all tiles.
def manhattan_distance(state):
    goal = [[1,2,3],
            [4,5,6],
            [7,8,0]]
    manhattan_dist = 0
    goal_pos={}
    for i in range(3):
        for j in range(3):
            tile = goal[i][j]
            #We create a dictionary,in order to store the goal position of each tile to look up.
            if tile != 0:
                goal_pos[tile] = (i,j)  #Dr. Eamonn mentioned that we should not count the blank.

    #Calculate the Manhattan distance.
    for i in range(3):
        for j in range(3):
            tile = state[i][j]
            if tile != 0:
                goal_i, goal_j = goal_pos[tile]
                manhattan_dist += abs(i - goal_i) + abs(j - goal_j) #In both vertical and horizontal direction,
we calculate the distance and add both up.
    return manhattan_dist

    '''
    #test

test = [        [1,2,3],
                [5,0,6],
                [4,7,8]]        #depth=4, using Dr. Eamonn's test case. Misplaced tile =4, manhattan distance =4.
print("Misplaced Tile Heuristic:", count_misplaced_tile(test))
print("Manhattan Distance Heuristic:", manhattan_distance(test))
test_2 = [          [1,2,3],
                    [4,5,6],
                    [0,7,8]]    #depth=2, using Dr. Eamonn's test case. Misplaced tile =2, manhattan distance =2.
```

```
print("Misplaced Tile Heuristic:", count_misplaced_tile(test_2))
print("Manhattan Distance Heuristic:", manhattan_distance(test_2))
'''
```

search.py

```
'''
Following Dr. Eamonn's pseudocode structure[1], with some variable name variations:

function general-search(problem, QUEUEING-FUNCTION)
nodes = MAKE-QUEUE(MAKE-NODE(problem.INITIAL-STATE))
loop do
    if EMPTY(nodes) then return "failure"
        node = REMOVE-FRONT(nodes)
    if problem.GOAL-TEST(node.STATE) succeeds then return node
        nodes = QUEUEING-FUNCTION(nodes, EXPAND(node, problem.OPERATORS))
end

[1]:
https://www.dropbox.com/scl/fo/blbkjaf1eyl94lij5wl2b/AAvNKn0YrnaX0oGPQn7ueFo?dl=0&e=2&rlkey=alq2gb2ftsw73hcar4lk
897r0  Dr. Eamonn's slide 2-Blind Search_part2.ppt, page 3
In my code:
- 'nodes' is implemented as 'frontier' (a list of nodes to be explored)
- 'node' is implemented as 'best' (the node with lowest f(n) value)
- 'problem.GOAL-TEST(node.STATE)' is implemented as 'check_is_goal()'
The core algorithm logic matches the pseudocode exactly.
'''

from node import Node
from puzzle import Puzzle
from heuristics import count_misplaced_tile, manhattan_distance
import time

def general_search_algorithm(puzzle_state, heuristic_function):
    start_time = time.time() #record the start time to calculate the running time of the algorithm.
    frontier = [] #I tried to use the heapq method but failed, so I chose to use a easier list to implement.
    visited = [] #record the visited states, to avoid repeated states.
    nodes_expanded = 0
    start_node = Node(puzzle_state, parent=None, g_n=0, h_n=heuristic_function(puzzle_state.state)) #initialize
the start node, and calculate its h_n value.
    frontier.append(start_node)
    max_queue_size = -1

    while len(frontier) > 0:
        if len(frontier) > max_queue_size:
            max_queue_size = len(frontier)#if update the max queue size.
        best = frontier[0]
        for node in frontier:
            if node.f_n < best.f_n:
                best = node         #update the best node in the frontier(has the lowest f_n value).
        frontier.remove(best)       #remove the best node from the frontier, and expand it.
        #We need to provide the traceback path in the report, so every time we expand a node, we need print the
path to reach it. We can trace back the path by using the parent.
        print(f"The best state to expand with g(n) = {best.g_n} and h(n) = {best.h_n} is...")
        best.state.display() #display the best node to expand.
        print()
        if best.state.check_is_goal():
            #we found it! return the best node, the number of nodes expanded and the max queue size. And in the
report, we can trace back the path and compare.
                print("Goal!!!")
                print(f"Solution depth was {best.g_n}")
                print(f"Number of nodes expanded: {nodes_expanded}")
                print(f"Max queue size: {max_queue_size}")
                end_time = time.time()
                print(f"Running time: {end_time - start_time:.1f} seconds")

                return best, nodes_expanded, max_queue_size
        #now mark it as visited.
        visited.append(best.tuple())
        nodes_expanded += 1

        children = best.expand() #Continue to expand the best node, and add its children to the frontier.

        for child in children:
            child.h_n = heuristic_function(child.state.state)
            child.f_n = child.g_n + child.h_n  #update the fn value.
```

```
            if child.tuple() not in visited:
                frontier.append(child)

    print("No solution is founded.")
    return None, nodes_expanded, max_queue_size #if we cannot find the solution, return None and the number of
nodes expanded and the max queue size.

def uniform_cost_search(puzzle_state):
    return general_search_algorithm(puzzle_state, heuristic_function=lambda state: 0)#As I mentioned in the
report, here we use a lambda function to make the heuristic value always 0.
def a_star_with_manhattan(puzzle_state):
    return general_search_algorithm(puzzle_state, heuristic_function=manhattan_distance)
def a_star_with_misplaced_tile(puzzle_state):
    return general_search_algorithm(puzzle_state, heuristic_function=count_misplaced_tile)
#call the heuristics functions from the heuristics.py file.
'''

#test

test_state = [  [1,2,3],
                [5,0,6],
                [4,7,8]]#depth=4, using Dr. Eamonn's test case.
puzzle = Puzzle(test_state)
result_node, nodes_expanded, max_queue_size = a_star_with_manhattan(puzzle)
print("Nodes Expanded:", nodes_expanded)
print("Max Queue Size:", max_queue_size)
#Output: Nodes Expanded: 4, Max Queue Size: 6
'''
```

node.py

```
#In the node file, we want to define the Node class for the search tree,
# and it is quite different from the state of the puzzle. But in the search tree,
# it contains the way for us to reach a puzzle state.
class Node:
    def __init__(self, state, parent=None, g_n=0, h_n=0):
        self.state = state
        self.parent = parent
        self.g_n = g_n                          # Cost to reach this node
        self.h_n = h_n                          # Heuristic cost to goal(estimate cost to reach goal)
        self.f_n = g_n + h_n                    # Total estimated cost
    def expand(self):
        # We want to expand the node to generate its children nodes.
        children = []
        row, col = self.state.find_blank()
        #try to move the blank in all four directions.
        try_right = self.state.move_right()
        if try_right is not None:
            right_node = Node(try_right, parent=self, g_n=self.g_n + 1)
            children.append(right_node)

        try_left = self.state.move_left()
        if try_left is not None:
            left_node = Node(try_left, parent=self, g_n=self.g_n + 1)
            children.append(left_node)

        try_up = self.state.move_up()
        if try_up is not None:
            up_node = Node(try_up, parent=self, g_n=self.g_n + 1)
            children.append(up_node)

        try_down = self.state.move_down()
        if try_down is not None:
            down_node = Node(try_down, parent=self, g_n=self.g_n + 1)
            children.append(down_node)

        #After trying all four directions, return the children list, so that we can pass it to the search algori
thm.
        return children

    #Dr.eamonn mentioned in the flie that repeated states should be avoided.
    #So we need to define the comparation method to compare two nodes.
    #I try to change the matrix state into a tuple, so that we can easily compare two states.
    def tuple(self):
        flat_state = []
        for row in self.state.state: #Sorry for the confusion, here self.state is a Puzzle object.
            for tile in row:
                flat_state.append(tile)
```

```
        return tuple(flat_state)
```

puzzle.py

```
class Puzzle:
    def __init__(self, state):
        # We want to initialize the puzzle, with a given state.
        self.state = state
        self.size = len(state)
    def find_blank(self):
        # We want to find the position of the 0(blank) in the puzzle.
        for i in range(self.size):
            for j in range(self.size):
                if self.state[i][j] == 0:
                    return (i, j)
        return None

    def copy(self):
        # We want to create a copy of the current puzzle state, to better compare states.
        state_copy = []
        for row in self.state:
            state_copy.append(row.copy())
        return state_copy

    def check_is_goal(self):
        # We want to check if the current state is the same with the goal state.
        goal = [[1,2,3],
                [4,5,6],
                [7,8,0]]
        return self.state == goal
        # Try to display the puzzle
    def display(self):
        for row in self.state:
            for num in row:
                if num == 0:
                    print("*", end=" ")
                else:
                    print(num, end=" ")
            print()

    #Instead of defining all 8 number's moves,
    #I noticed that we can just define the blank's move
    #(Because every move can be seen as the number swop with the blank).

    def move_up(self):
        blank_position = self.find_blank()
        row = blank_position[0]
        col = blank_position[1]
        #check whether we can move up or not, if not, return None.
        if row == 0:
            return None
        updated_state = self.copy()
        updated_state[row][col]= updated_state[row-1][col]
        updated_state[row-1][col]=0
        return Puzzle(updated_state)
    #define move in other directions in the same way.
    def move_down(self):
        blank_position = self.find_blank()
        row = blank_position[0]
        col = blank_position[1]
        if row == self.size - 1:
            return None
        updated_state = self.copy()
        updated_state[row][col] = updated_state[row + 1][col]
        updated_state[row + 1][col] = 0
        return Puzzle(updated_state)
    def move_right(self):
        blank_position = self.find_blank()
        row = blank_position[0]
        col = blank_position[1]
        if col == self.size - 1:
            return None
        updated_state = self.copy()
        updated_state[row][col] = updated_state[row][col + 1]
        updated_state[row][col + 1] = 0
        return Puzzle(updated_state)
```

```
def move_left(self):
    blank_position = self.find_blank()
    row = blank_position[0]
    col = blank_position[1]
    if col == 0:
        return None
    updated_state = self.copy()
    updated_state[row][col] = updated_state[row][col - 1]
    updated_state[row][col - 1] = 0
    return Puzzle(updated_state)
```